

Project Plan

Versione	Data	Descrizione
1.0	24 Settembre 2026	Prima versione del documento Project Plan

1.Introduzione

Il presente documento descrive il piano di progetto per lo sviluppo di un gioco di carte collezionabili (CCG) multiplayer online di nome **Thearchis**. Il progetto nasce dall'esigenza dell'azienda proprietaria di offrire agli utenti un'esperienza di gioco strategica, competitiva e altamente coinvolgente. Il prodotto combinerà meccaniche di deck-building (costruzione del mazzo) con un sistema di matchmaking e gestione in tempo reale delle partite multiplayer. Gli utenti potranno accedere alla propria collezione e ai loro dati grazie ad un database.

1.1 Obiettivi del progetto

- **Coinvolgimento e fidelizzazione dell'utente:** offrire una piattaforma Free-to-Play in cui i giocatori possono collezionare carte, testare diverse strategie ed evolvere le proprie abilità
- **Competitività Multiplayer:** garantire un'infrastruttura di rete solida e bilanciata per consentire partite multigiocatore in tempo reale

1.2 Responsabili del progetto

- **Stakeholder:** società proprietaria di *Thearchis*
- **Team di sviluppo:** Il team responsabile del ciclo di vita del software si chiama come il gioco, *Thearchis*, ed è composto da Alessandro Cardillo Ottaviano

2.Modello di processo

Il team *Thearchis* adotta il framework agile *Scrum*, integrato con la metodologia *Kanban*, per garantire uno sviluppo software incrementale, adattivo e focalizzato sul rilascio continuo di valore. Il ciclo di vita del progetto si articola in iterazioni disciplinate dalle seguenti fasi e artefatti:

- **Product Backlog:** elenco dinamico e prioritizzato di tutti i requisiti di progetto, mantenuto e aggiornato continuamente dal *Product Owner*. È suddiviso in:
 - Requisiti funzionali: caratteristiche del gioco (es. logica di scontro, gestione del mazzo, interfaccia utente)
 - Requisiti non funzionali: prestazioni di rete, sicurezza, usabilità e vincoli architetturali
 - Bug e problemi: correzione di difetti emersi durante lo sviluppo

- **Sprint Planning:** riunione di avvio della *Sprint* in cui *Product Owner* e *team di Sviluppo* selezionano gli item prioritari dal *Product Backlog*, definiscono lo *Sprint Goal* (obiettivo dello sprint) e formalizzano lo *Sprint Backlog* con le relative stime operative
- **Sprint:** periodo di lavoro in cui il team trasforma gli item dello *Sprint Backlog* in un incremento finito, testato e funzionante
- **Daily Scrum:** breve incontro quotidiano focalizzato sull'allineamento del team per:
 - Condividere l'avanzamento rispetto allo *Sprint Goal*
 - Identificare tempestivamente eventuali ostacoli o impedimenti
 - Pianificare le attività prioritarie per la giornata
- **Sprint Review:** incontro al termine delle *Sprint* in cui il team presenta l'incremento di prodotto al committente/stakeholder per raccogliere feedback sul software. Il *Product Owner* valuta la conformità del lavoro svolto e integra i feedback nel *Product Backlog* per i cicli successivi
- **Sprint Retrospective:** incontro di valutazione interna focalizzato esclusivamente sull'ottimizzazione del processo di lavoro. Il team analizza cosa ha funzionato, cosa può essere migliorato e definisce azioni concrete per lo *Sprint* successivo

Per affiancare la struttura di *Scrum*, il team utilizza una **Kanban Board** (*GitHub Projects*). Questo strumento permette di visualizzare lo stato di ogni singola attività in tempo reale, monitorare il carico di lavoro del team e limitare i colli di bottiglia durante lo sviluppo.

3. Organizzazione del processo

L'organizzazione del team *Thearchis* si basa sui ruoli previsti dal framework *Scrum*. Questo garantisce l'auto-organizzazione, la tracciabilità delle responsabilità e una netta separazione tra la visione strategica di prodotto e l'esecuzione tecnica.

3.1 Ruoli nel progetto

- **Product Owner (PO):** rappresenta la voce degli stakeholder e garantisce che lo sviluppo risponda al valore di business richiesto. Definisce la visione del gioco, mantiene, architetta e ordina la priorità degli item nel *Product Backlog*, definisce i criteri di accettazione e convalida gli incrementi software al termine di ogni *Sprint*
- **Scrum Master (SM):** agisce come facilitatore del processo, assicurando la corretta applicazione della metodologia *Scrum* e promuovendo l'efficienza del team. Organizza e modera le cerimonie *Scrum*, rimuove tempestivamente gli ostacoli operativi o tecnici e guida il miglioramento continuo del processo di lavoro
- **Team di Sviluppo:** unità multidisciplinare e auto-organizzata che si fa carico della trasformazione dei requisiti in un incremento di prodotto testato e funzionante. Scompone i requisiti in task tecnici, stima lo sforzo necessario, scrive il codice sorgente, esegue i test unitari/di integrazione, cura la documentazione e gestisce il versionamento su *GitHub*

4. Standard, linee guida e procedure

4.1 Standard di Sviluppo e Qualità

- **Convenzioni di codice Java:** il codice deve rispettare rigorosamente le convenzioni standard di sintassi e stile di *Java* (*Oracle Naming Conventions* e principi di *Clean Code*) per garantire leggibilità, uniformità e manutenibilità del sistema
- **Gestione del versionamento:** l'intero codice sorgente è gestito tramite *GitHub*. È vietato il commit diretto sul ramo principale (*main*); ogni attività deve essere svolta all'interno di un ramo (*branch*) dedicato
- **Controllo qualità:** ogni incremento software deve superare una suite di test unitari e di integrazione per garantire che le nuove funzionalità non compromettano la stabilità del sistema esistente

4.2 Linee Guida Operative

- **Gestione delle attività:** il flusso di lavoro viene monitorato in tempo reale tramite una *Kanban Board* su *GitHub Projects*, strutturata per tracciare lo stato dei task (*To Do*, *In Progress*, *Done*) e distribuire in modo equilibrato il carico di lavoro

4.3 Procedure Operative

- **Integrazione del codice (Merge):** ogni nuova funzionalità viene sviluppata su un *branch* separato. Il *merge* sul ramo principale (*main*) è consentito unicamente previa verifica del corretto funzionamento e superamento dei test previsti, gestito tramite *Pull Request* su *GitHub*
- **Segnalazione e gestione bug:** qualsiasi errore, anomalia o malfunzionamento del software deve essere tracciato mediante l'apertura di una *Issue* su *GitHub*, specificando la descrizione del problema ed il livello di priorità
- **Validazione tramite test:** l'esecuzione dei test (unitari e di integrazione) è richiesta in due momenti chiave: ad ogni incremento significativo all'interno del *branch* di sviluppo e come controllo bloccante prima di effettuare il *merge* sul ramo *main*.

5. Attività di gestione

La gestione del progetto segue un ciclo iterativo e ricorsivo strutturato secondo le fasi del framework *Scrum*. Ciascuna iterazione di sviluppo si articola nella seguente sequenza cronologica di attività:

- **Pianificazione dell'iterazione (Product Backlog & Sprint Planning):** svolta all'inizio di ogni ciclo. Il *Product Owner* e il *team di sviluppo* selezionano, prioritizzano e stimano gli item dal *Product Backlog*, definendo lo *Sprint Goal* e lo *Sprint Backlog* per l'iterazione imminente
- **Esecuzione dello Sprint:** finestra temporale dalla durata variabile (1 o 2 settimane), durante la quale il team lavora alla trasformazione dei requisiti scelti in un incremento software o documentale testato e funzionante

- **Allineamento operativo (Daily Scrum simulata):** incontri di coordinamento breve svolti giornalmente per monitorare l'avanzamento dei task, individuare tempestivamente eventuali impedimenti e riallineare le attività operative
- **Chiusura e Valutazione (Sprint Review & Sprint Retrospective):** cerimonie tenute al termine di ogni *Sprint* dedicate a due momenti distinti:
 - **Sprint Review:** presentazione dell'incremento completato agli stakeholder per la convalida del lavoro e la raccolta di feedback sul prodotto
 - **Sprint Retrospective:** analisi interna del processo di lavoro per identificare punti di forza, criticità e azioni di miglioramento metodologico da applicare nello *Sprint* successivo

6. Rischi potenziali

6.1 Rischi Tecnici e di Competenze

- **Competenza limitata sui framework GUI e sulla gestione di rete (multiplayer):** la scarsa familiarità iniziale con i framework per l'interfaccia grafica (*JavaFX*) e con l'architettura di rete in tempo reale (Socket/Protocolli di comunicazione) potrebbe rallentare lo sviluppo
- **Complessità nell'interazione tra le carte e regole di gioco:** gestire gli effetti speciali delle carte, i turni e le priorità d'azione può generare una logica di codice complessa e difficile da mantenere o testare. Verranno definiti test unitari (*JUnit*) dedicati alla validazione di ogni singola regola prima di integrarla

6.2 Rischi di Pianificazione e Processo

- **Sottostima dello sforzo operativo e accumulo di requisiti:** una stima errata dei tempi o l'aggiunta in itinere di nuove funzionalità possono sovraccaricare gli *Sprint*, compromettendo il rilascio dell'incremento. Per evitare ciò ogni item del *Product Backlog* verrà pesato in funzione della sua complessità/tempo necessario di sviluppo, ed ogni *Sprint* avrà un limite massimo di items

7. Personale

RUOLO	PERSONALE	RESPONSABILITÀ
Scrum Master	Alessandro Cardillo Ottaviano	Facilita e garantisce il processo <i>Scrum</i> promuovendo il miglioramento del team.
Product Owner	Alessandro Cardillo Ottaviano	Simulare i bisogni e le aspettative del cliente compilando il <i>Product backlog</i> e gestendo le priorità

		dei requisiti. Aggiornare questo documento in base allo stato dello sviluppo.
Team di sviluppo	Alessandro Cardillo Ottaviano	Redigere la documentazione necessaria. Sviluppare e testare il codice. Modellare con UML.

8. Metodi e tecniche

8.1 Ingegneria dei Requisiti

- **Estrapolazione requisiti:** i requisiti funzionali saranno derivati dalle regole e meccaniche del gioco e integrati dal *Product Owner*
- **Criteri di accettazione:** ogni requisito sarà associato a precisi criteri di accettazione per verificarne l'effettivo completamento durante la fase di test
- **Prioritizzazione MoSCoW:** l'assegnazione delle priorità ai requisiti seguirà il modello *MoSCoW*:
 - **M (Must have):** requisiti indispensabili per l'architettura base
 - **S (Should have):** funzionalità importanti per il gioco, ma non critiche per il rilascio iniziale
 - **C (Could have):** caratteristiche migliorative o estetiche da implementare solo in caso di risorse residue
 - **W (Won't have):** requisiti esplicitamente esclusi dal perimetro del progetto
- **Stima dello sforzo:** a ciascun requisito verrà assegnato un peso in termini di carico di lavoro e complessità

8.2 Progettazione Architettuale

- **Progettazione incrementale:** l'architettura del sistema non sarà definita in modo monolitico iniziale, ma evolverà in modo incrementale a ogni *Sprint*
- **Modellazione UML:** utilizzo del linguaggio UML per rappresentare la struttura e le interazioni del sistema
- **Strumenti di modellazione:**
 - **Papyrus:** utilizzato per la stesura del *Class Diagram*
 - **LucidChart:** utilizzato per la creazione degli altri diagrammi di supporto

8.3 Implementazione e Gestione della Configurazione

- **Sviluppo Incrementale:** ogni iterazione produce un incremento di codice funzionante, integrato e pronto per la dimostrazione durante la *Sprint Review*
- **Versionamento:** gestione della configurazione e del codice sorgente affidata a **GitHub**, sfruttando la struttura a *branch* per isolare lo sviluppo delle singole funzionalità

- **Build e dipendenze:** adozione di **Apache Maven** per l'automazione della build, la gestione del ciclo di vita del software e l'importazione delle librerie esterne.
- **Interfaccia Grafica (GUI):** sviluppo del *front-end* e delle componenti visuali di gioco mediante il framework **JavaFX**
- **Analisi statica e qualità del codice:** mantenimento degli standard di qualità tramite l'uso di plugin integrati nell'IDE:
 - **EclipsePMD:** per l'individuazione di code smell, bug potenziali e violazioni delle convenzioni di programmazione
 - **Stan4J:** per l'analisi della complessità strutturale, dell'accoppiamento e delle dipendenze tra i pacchetti Java

8.4 Testing e Ambiente di Prova

- **Ambiente di prova:** l'ambiente di test e le apparecchiature di verifica coincidono con le macchine locali di sviluppo della squadra
- **Esecuzione dei test:** i test e il collaudo avverranno in ambiente di sviluppo **Eclipse IDE**: dalla validazione della logica di dominio back-end (regole di gioco, gestione mazzi e turni) fino alle componenti di interfaccia utente
- **Framework di testing:** utilizzo di **JUnit** per la scrittura ed esecuzione automatizzata dei test unitari e di integrazione legati ai casi d'uso del sistema

9. Garanzia di qualità

La *Garanzia di Qualità* (QA) definisce l'insieme delle attività e delle responsabilità volte ad assicurare che il software sviluppato rispetti i requisiti funzionali, i requisiti non funzionali e gli standard qualitativi concordati. Nel contesto del processo *Scrum*, la qualità è una responsabilità condivisa tra tutti i ruoli del team.

9.1 Responsabilità per la Qualità

- **Product Owner (qualità di prodotto):**
 - definisce e formalizza i requisiti funzionali e non funzionali che stabiliscono lo standard qualitativo del gioco
 - specifica i criteri di accettazione per ogni requisito del *Product Backlog*
 - convalida il software durante la *Sprint Review*, verificando che l'incremento risponda esattamente alle aspettative del committente
- **Development Team (qualità tecnica e del codice):**
 - garantisce la qualità del codice sorgente rispettando le convenzioni ufficiali di programmazione *Java* e i principi di *Clean Code*
 - esegue regolarmente i test unitari e d'integrazione (tramite **JUnit**) prima di ogni operazione di *merge* sul ramo principale
 - utilizza gli strumenti di analisi statica del codice (**EclipsePMD** e **Stan4J**) per identificare ed eliminare tempestivamente code smell, vulnerabilità e criticità architetturali
- **Scrum Master (qualità del processo):**

- monitora il rispetto della metodologia *Scrum*, delle linee guida operative e delle procedure concordate dal team
- assicura che nessun incremento venga considerato completato senza aver superato tutti i controlli qualitativi
- rimuove gli ostacoli procedurali o organizzativi che potrebbero compromettere la qualità del lavoro svolto

10. Work packages

I Work Packages sono organizzati secondo un'architettura a **progetto Multi-Modulo Maven**. Questa struttura garantisce la separazione delle responsabilità, semplifica la gestione delle attività nel team e assicura che il pacchetto distribuito al giocatore finale (*Client*) sia del tutto privo del codice sorgente relativo al *Server* e al *Database*, preservando la sicurezza del sistema.

- Modulo **Shared (game-common)** – logica di gioco e dominio: contiene le entità e la logica di base condivise sia dal *Client* che dal *Server*
 - Modellazione del dominio: definizione delle classi per carte, mazzi, mano del giocatore e stato del campo di gioco
 - Regolamento: algoritmi per la gestione dei turni, calcolo dei danni e risoluzione delle abilità delle carte
 - Protocollo di rete: definizione dei modelli e del formato dei messaggi scambiati via Socket tra *Client* e *Server*
 - Testing: suite di test unitari dedicati alla validazione preliminare delle regole
- Modulo **Client (game-client)** – interfaccia grafica ed esperienza utente: modulo dedicato all'applicazione riservata al giocatore finale, compilato come eseguibile indipendente
 - Interfaccia grafica (*JavaFX*): progettazione dei layout per il menu principale, la creazione del mazzo, la gestione della collezione, la lobby ed il tavolo da gioco
 - Interattività: gestione degli input dell'utente (utilizzo carte, pulsanti d'azione) e rendering grafico delle animazioni
 - Connessione client: gestione del socket lato client per l'invio e la ricezione dei comandi al server in tempo reale
- Modulo **Server (game-server)**
 - Infrastruttura multiplayer: modulo backend responsabile dell'arbitraggio delle partite e della gestione delle connessioni di rete
 - Server socket: architettura per la gestione della concorrenza e l'ascolto delle connessioni in arrivo
 - matchmaking e lobby: creazione delle stanze da gioco e abbinamento dei giocatori
 - Sincronizzazione: arbitraggio dello stato di gioco e sincronizzazione dei turni in tempo reale per prevenire manomissioni o desincronizzazioni lato client
 - Persistenza e database: componente integrata nel backend per la gestione permanente delle informazioni degli utenti

- Accesso ai dati: implementazione dei livelli di lettura/scrittura sul *Database*
- Gestione utenze: autenticazione, registrazione dei giocatori e tracciamento dello storico delle partite
- Salvataggio dati: validazione lato *Server* e salvataggio dei mazzi personalizzati creati dagli utenti

11. Risorse

La realizzazione del progetto *Thearchis* si avvale di una combinazione di risorse hardware e infrastrutturali, strumenti software e competenze umane distribuite all'interno del team.

11.1 Risorse Hardware e Infrastruttura Cloud

- **Stazioni di lavoro locali:** ogni membro del team dispone di un proprio calcolatore dedicato alle attività di modellazione, scrittura del codice, debugging e testing unitario locale
- **Server in cloud:** infrastruttura cloud dedicata all'hosting ed esecuzione continua del modulo *backend* “*game-server*”, per garantire la gestione fluida e remota delle sessioni di gioco multiplayer
- **Database in cloud:** Istanza di *database* ospitata su servizio cloud per consentire l'accesso remoto, centralizzato e sicuro ai dati di gioco (profili utente, salvataggio dei mazzi e della collezione)

11.2 Risorse Software e Strumenti di Sviluppo

- **Gestione del codice e del progetto:** **GitHub** come repository centrale per il controllo di versione, la gestione dei *branch*, il tracciamento delle *Issue* e la gestione operativa tramite *Kanban Board*
- **Ambiente di sviluppo e build:** **Eclipse IDE** come ambiente integrato principale per la programmazione in *Java*; **Apache Maven** per la gestione automatizzata delle dipendenze e la compilazione della struttura multi-modulo
- **Modellazione:** **Papyrus** per la modellazione del *Class Diagram*
- **Interfaccia grafica:** **JavaFX** per la realizzazione dell'interfaccia utente grafica e la gestione delle componenti visive del *client*

11.3 Risorse Umane

- **Team *Thearchis*:** il team opera in modo autonomo e multidisciplinare, coprendo collettivamente tutti i ruoli del framework *Scrum* (*Product Owner*, *Scrum Master*, *Development team*) ed eseguendo l'intero ciclo delle attività di analisi, progettazione, sviluppo, testing e documentazione

12. Budget e programma

12.1 Budget Economico

Trattandosi di un progetto universitario senza scopo di lucro, il budget finanziario assegnato all'iniziativa è pari a 0 €:

- **Strumenti e licenze:** l'intero stack tecnologico scelto (*Java*, *JavaFX*, *Eclipse*, *Maven*, *Papyrus*, *JUnit*, *GitHub*) è costituito da software open-source o distribuito con licenze comunitarie/gratuite
- **Infrastruttura:** l'hardware di sviluppo è rappresentato dai calcolatori personali dei membri del team, mentre le risorse di hosting cloud (server e database) faranno ricorso ai piani gratuiti messi a disposizione dai fornitori di servizi cloud

12.2 Programma di Sviluppo e Time Budget

La scansione temporale delle attività non segue un modello rigido a cascata, ma si basa sul timeboxing fornito dal framework *Scrum*.

13. Cambiamenti

Trattandosi di un progetto gestito con metodologia Agile (*Scrum*), l'intero processo di sviluppo è intrinsecamente strutturato per accogliere ed integrare i cambiamenti in modo controllato, anziché contrastarli.

13.1 Modifiche ai Requisiti e al Perimetro di Progetto

- **Responsabilità del Product Owner:** ogni nuova richiesta di funzionalità, modifica a meccaniche di gioco esistenti o riprioritizzazione è di esclusiva pertinenza del *Product Owner*
- **Aggiornamento del Product Backlog:** i nuovi requisiti o le variazioni vengono formalizzati sotto forma di *User Story*, inseriti nel *Product Backlog*, valutati e prioritizzati (tramite il modello *MoSCoW*)
- **Pianificazione negli Sprint:** per preservare la stabilità del lavoro del team, un requisito modificato non stravolge mai lo *Sprint Backlog* in corso; viene invece pianificato ed allocato in uno degli *Sprint* successivi durante la consueta fase di *Sprint Planning*

13.2 Modifiche al Codice Sorgente e alla Documentazione

- **Controllo di versione:** tutta la tracciabilità delle modifiche al codice e ai file di documentazione è garantita dal sistema di versionamento su *GitHub*
- **Strategia di branching:** lo sviluppo di nuove funzionalità o la risoluzione di bug avviene su rami di lavoro dedicati (*branch*), isolandoli dal codice stabile

- **Integrazione e code review:** l'unione delle modifiche nel ramo principale (*main*) avviene esclusivamente tramite *Pull Request*, garantendo che ogni cambiamento sia revisionato e testato prima di diventare definitivo

14. Consegna

Il prodotto finale verrà reso disponibile come file eseguibile pronto per l'utente.

[Da concordare]